

Project Description – Personal Health Tracker

Phase 1: Problem Understanding & Industry Analysis

1.Requirement Gathering

Users should be able to register their health consultation request by providing:

- Full Name
- Age
- Contact Number
- Email Address
- Health Issues / Symptoms
- Preferred Appointment Slot
- Assigned Nutritionist
- Default Request Status: *Pending*

2.Stakeholder Analysis

- **Users :**
 - Submit their consultation requests with personal and health details.
- **Support Staff (Attendants):**
 - Access all consultation requests submitted by users.
 - View users' past health/consultation records.
 - Update status as the consultation progresses (Pending → Confirmed → Completed).
 - Ensure that users receive email notifications whenever the status is changed.
- **Nutritionists:**

- View only the confirmed consultation requests assigned to them.
- Add personalized notes, diet charts, or lifestyle suggestions after the consultation.

3.Business Process Flow

- A user places a consultation request → Request status is set as Pending.
- Attendant reviews the request → Can refer to historical records if available → Updates the status accordingly.
- System sends **automated email alerts** to users whenever there is a status update.
- Nutritionist accesses the confirmed requests and provides consultation notes or diet plans.

4.Industry Use Case Analysis

- Maintains accurate **health records and consultation history** for each user.
- Facilitates nutritionists and staff in making **better health and lifestyle recommendations** by referring to previous data.
- Improves **timely communication and transparency** between users, attendants, and nutritionists.

5.AppExchange Exploration

- Review Salesforce healthcare/wellness apps that support **health management**.
- Identify useful components such as:
 - Automated email reminders
 - Appointment and scheduling tools
 - Case/record management features

- Health history storage
- Personalized nutrition and wellness tracking

Phase 2: Org Setup & Configuration – Personal Health Tracker

1) Salesforce Edition

- A **Salesforce Developer Org** was created using developer.salesforce.com/signup.
- This environment is used for development, testing, and configuration.

2) Basic Company Setup

- From the **Setup** (⚙ icon) → **Company Information**, the basic company details were configured.

The screenshot shows the Salesforce Setup interface for an organization named 'Personal Health Tracker'. The left sidebar contains navigation links for 'Setup', 'Home', and 'Object Manager'. The main content area is titled 'Company Information' and includes a search bar for 'company info'. Below this, there are sections for 'General Information', 'Address', and 'Locale Settings'. The 'General Information' section contains fields for 'Organization Name' (Personal Health Tracker), 'Primary Contact' (Sapna Singh), 'Division', 'Phone', and 'Fax'. The 'Address' section includes fields for 'Country' (India), 'Street', 'City' (Amravati), 'State/Province' (Maharashtra), and 'Zip/Postal Code' (444607). The 'Locale Settings' section shows 'Default Locale' (English (India)) and 'Default Language' (English). A 'Required Information' indicator is visible in the top right corner of the form.

3) Creation of Lightning App

Steps:

1. Navigate to **Setup** → **Quick Find** → **App Manager** → *New Lightning App*.
2. Configure the app as per project needs.

The screenshot shows the 'App Settings' page in the Lightning App Builder. The left sidebar has 'App Settings' selected, with sub-items: 'App Details & Branding', 'App Options', 'Utility Items (Desktop Only)', 'Navigation Items', and 'User Profiles'. The main content area is titled 'App Details & Branding' and includes instructions: 'Give your Lightning app a name and description. Upload an image and choose the highlight color for its navigation bar.' It is divided into two columns: 'App Details' and 'App Branding'. In 'App Details', there are fields for 'App Name' (Personal Health Tracker), 'Developer Name' (Personal_Health_Tracker), and 'Description' (Tracks user health logs with admin oversight). In 'App Branding', there is an 'Image' field with a placeholder logo, a 'Primary Color Hex Value' field (set to #007002), and an 'Org Theme Options' checkbox (unchecked). Below these is an 'App Launcher Preview' showing the app's icon and name.

Fiscal Year Settings

- **Purpose:** Defines reporting periods for dashboards and reports.
- **Type Used:** Standard Fiscal Year (Jan–Dec).

The screenshot shows the 'Fiscal Year' setup page in Salesforce. The top navigation bar includes the 'PERSONAL HEALTH TRACKER' logo, a search bar, and various utility icons. The left sidebar has 'Setup' selected, with 'Fiscal Year' highlighted under 'Company Settings'. The main content area is titled 'Organization Fiscal Year Edit: Personal Health Tracker'. It includes a 'Fiscal Year Information' section with a warning about changing the fiscal year and a 'Change Fiscal Year Period' section. The 'Change Fiscal Year Period' section has a table with the following data:

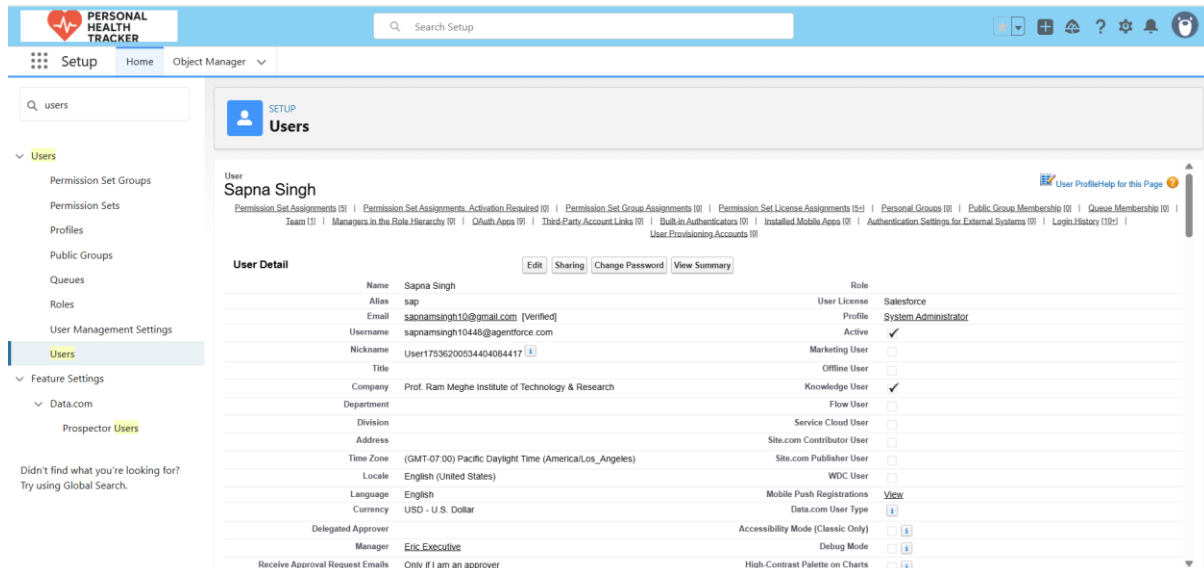
Name	Fiscal Year Start Month	Fiscal Year is Based On
Personal Health Tracker	January	☒ The ending month ☐ The starting month

The URL at the bottom of the page is: <https://orgfarm-f156258042-dev-ed.develop.lightning.force.com/lightning/setup/ForecastFiscalYear/home>

4) User Setup & Licenses

- **Users:** Every nutritionist, attendant, or admin requires a Salesforce user account.
- **Licenses:**
 - *Salesforce License* assigned based on the sub-profile (Nutritionist/Admin).

Project Relevance: Helps track consultations and system activity over a defined fiscal period.



5) Profiles

- **Definition:** Profiles manage object-level and field-level access (CRUD).
- **Project Profiles:**
 - *User Profile:* Can create and view only their own health records.
 - *Nutritionist Profile:* Can access all patient records and update consultation notes.
 - *Attendant Profile:* Can update consultation status but cannot edit nutritionist notes.

👉 CRUD permissions are applied according to each sub-profile.

6) Roles

- **Definition:** Role hierarchy controls record visibility.

- **Project Setup:**

- Not required here, since record visibility is fully managed through sub-profiles (User, Nutritionist, Attendant).

7) Permission Sets

- **Definition:** Provide additional permissions beyond profile settings.
- **Project Usage:**
 - Permission sets were not used.
 - Instead, all required access was handled through **sub-profiles**, ensuring users only have permissions relevant to their role.

8) Organization-Wide Defaults (OWD)

- **Definition:** Sets baseline record access for all users.
- **Configuration:**
 - *User__c*: Default OWD → Patients see only their own records. Nutritionists and attendants see records according to their sub-profile access.
 - *Consultation__c* / *Consultation_Notes__c*: OWD set to **Private**. Access is controlled by sub-profile rules, allowing nutritionists to update consultations and patients to see only their own details.

PERSONAL HEALTH TRACKER Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

User

Details

Fields & Relationships 14 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Age	Age__c	Number(3, 0)		
Appointment Slot	Appointment_Slot__c	Date/Time		
Contact Number	Contact_Number__c	Phone		
Created By	CreatedById	Lookup(User)		
Currency	CurrencyIsoCode	Picklist		
Email Address	Email_Address__c	Email		
Full Name	Full_Name__c	Text(50)		
Health Issues	Health_Issues__c	Long Text Area(32768)		
Last Modified By	LastModifiedById	Lookup(User)		

Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access
Triggers

PERSONAL HEALTH TRACKER Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

Nutritionist

Details

Fields & Relationships 10 Items, Sorted by Field Label

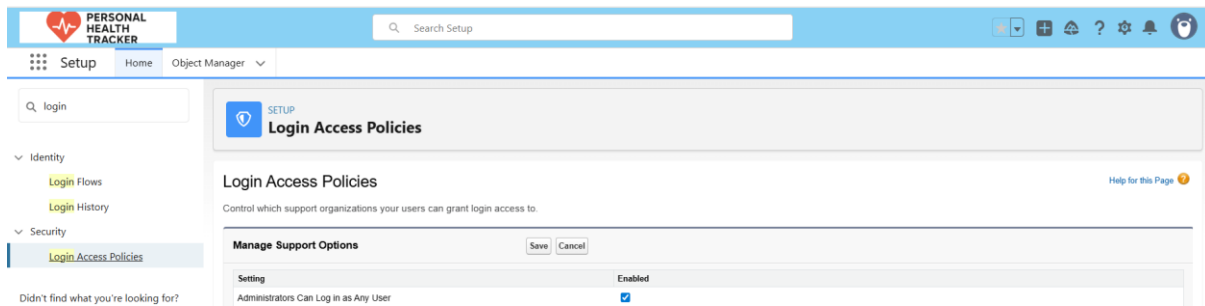
Quick Find New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Contact Number	Contact_Number__c	Phone		
Created By	CreatedById	Lookup(User)		
Currency	CurrencyIsoCode	Picklist		
Email Address	Email_Address__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Nutritionists Name	Nutritionists_Name__c	Text(50)		
Nutritionists Notes Name	Name	Text(80)		✓
Owner	OwnerId	Lookup(User,Group)		✓
Specialization	Specialization__c	Picklist		
User	User__c	Lookup(User)		

Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access
Triggers

9) Login Access Policies

- **Purpose:** Defines how users can log into the system.
- **Relevance:** Admins can log in for troubleshooting when required.



10) Development Org Setup

- **Purpose:** Prepare the development environment.
- **Project Tasks:**
 - Created custom objects → User, Consultation, Notes.
 - Added key fields such as *Symptoms__c*, *Preferred_Consultation_Time__c*, *Nutritionist_Notes__c*.

11) Sandbox Usage

- For this project, the **Free Developer Edition** served as the testing environment.
- All Lightning Web Components (LWCs), Apex classes, and triggers were created and tested here before being finalized.
- Ensures patient management, status updates, and email alerts work as expected prior to production use.

12) Deployment Basics

- Since development was done directly in the **Developer Edition**, formal **Change Sets** were not needed.
- LWCs were deployed using the **LWC Editor / Salesforce Extension**.

- All objects, fields, triggers, and LWCs were built and tested directly in the same environment.
- Permissions were managed via **user sub-profiles**, so no extra profile adjustments were required at deployment time.

Phase 3: Data Modeling & Relationships

1. Create User__c Object

Step 1: Create Object

1. Go to **Setup (⚙) → Object Manager → Create → Custom Object**.
2. Fill details:
 - **Label:** User
 - **Plural Label:** Users
 - **Object Name:** User
 - **Record Name:** User Name → Data Type: Text
 - Check: **Allow Reports, Allow Activities, Track Field History**
 - Save.

The screenshot shows the Salesforce Setup interface for the 'User' object. The left sidebar contains a list of configuration options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, Object Access, and Triggers. The main content area is titled 'Edit Custom Object User' and features the 'Custom Object Definition Edit' form. This form includes a 'Custom Object Information' section with the following fields: 'Label' (set to 'User'), 'Plural Label' (set to 'Users'), 'Object Name' (set to 'User'), and a 'Description' text area. There are also checkboxes for 'Starts with vowel sound' and 'Context-Sensitive Help Setting'. The form has 'Save', 'Save & New', and 'Cancel' buttons at the top right. A red error message 'Required Information' is visible next to the 'Label' field.

Step 2: Create Fields

Go to **User__c** → **Fields & Relationships** → **New**. Add the following:

1. Age__c

- Data Type: Number
- Length: 3, Decimal: 0
- Label: Age
- Save

2. Mobile__c

- Data Type: Phone
- Label: Mobile Number
- Save

3. Email__c

- Data Type: Email
- Label: Email ID
- Save

4. History__c

- Data Type: Long Text Area
- Label: Health History
- Length: 32000, Visible Lines: 5
- Save

5. Salesforce_User__c

- Data Type: Lookup Relationship
- Related To: User
- Label: Related Salesforce User
- Save

6. Status__c

- Data Type: Picklist
- Values: Pending, Confirmed, Completed, Cancelled
- Save

SETUP > OBJECT MANAGER

User

Details

Fields & Relationships
15 Items, Sorted by Field Label

Q Quick Find New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Age	Age__c	Number(3, 0)		▼
Appointment Slot	Appointment_Slot__c	Date/Time		▼
Contact Number	Contact_Number__c	Phone		▼
Created By	CreatedById	Lookup(User)		
Currency	CurrencyIsoCode	Picklist		▼
Email Address	Email_Address__c	Email		▼
Full Name	Full_Name__c	Text(50)		▼
Health Issues	Health_Issues__c	Long Text Area(32768)		▼
Last Modified By	LastModifiedById	Lookup(User)		

Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access
Triggers

2. Create Nutritionist__c Object

Step 1: Create Object

- Label: Nutritionist

- Plural: Nutritionists
- Record Name: Nutritionist Name (Text)
- Enable Reports, Activities, Track Field History
- Save

The screenshot shows the Salesforce Setup interface for a custom object named 'Nutritionist'. The left sidebar contains a navigation menu with options like 'Details', 'Fields & Relationships', 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', 'Object Limits', 'Record Types', 'Related Lookup Filters', 'Restriction Rules', 'Scoping Rules', and 'Object Access'. The main content area is titled 'Edit Custom Object Nutritionist' and includes a 'Custom Object Definition Edit' section. This section contains fields for 'Label' (Nutritionist), 'Plural Label' (Nutritionists), 'Object Name' (Nutritionist), and a 'Description' text area. There are also checkboxes for 'Starts with vowel sound' and 'Context Sensitive Help Setting'. The bottom of the page has a 'Enter Record Name Label and Format' section.

Step 2: Create Fields

1. Specialization__c

- Data Type: Picklist
- Values: Diet Planning, Weight Management, Sports Nutrition, Pediatric Nutrition, Clinical Nutrition, Others
- Save

2. Phone__c

- Data Type: Phone
- Label: Phone
- Save

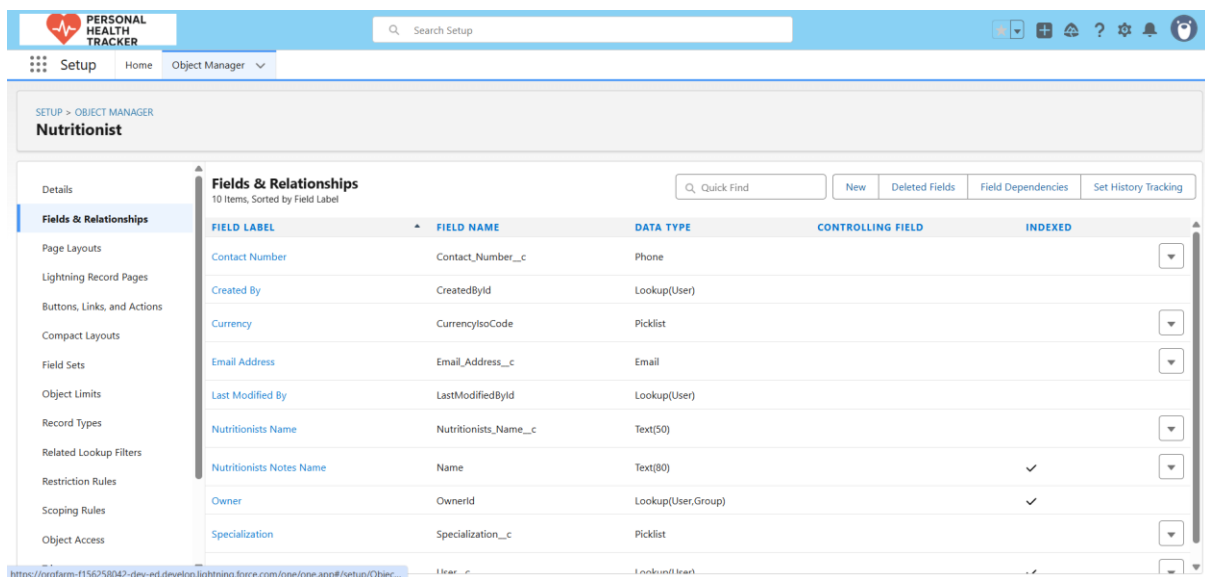
3. Email__c

- Data Type: Email

- Label: Email
- Save

4. Salesforce_User__c

- Data Type: Lookup Relationship
- Related To: User
- Label: Related Salesforce User
- Save



3. Create Consultation__c Object

Step 1: Create Object

- Label: Consultation
- Plural: Consultations
- Record Name: Consultation Name (Auto Number if required)
- Enable Reports, Activities, Track Field History
- Save

The screenshot shows the Salesforce 'New Custom Object' setup page. The header includes the 'PERSONAL HEALTH TRACKER' logo and a search bar. The main content area is titled 'New Custom Object' and contains a 'Custom Object Definition Edit' section. This section includes fields for 'Label' (Consultation), 'Plural Label' (Consultations), 'Object Name' (Consultation), and a 'Description' text area. There are also checkboxes for 'Starts with vowel sound' and 'Context Sensitive Help Setting'. A 'Required Information' icon is visible on the right side of the form.

Step 2: Create Fields

1. **Symptoms__c** – Long Text Area
2. **Preferred_Consultation_Time__c** – Date/Time
3. **Status__c** – Picklist (Pending, Confirmed, Completed, Cancelled, default: Pending)
4. **User__c** – Master-Detail → Related To: User__c
5. **Nutritionist__c** – Lookup → Related To: Nutritionist__c
6. **Created_By__c** – Lookup → Related To: Salesforce User

4. Create ConsultationNote__c Object

Step 1: Create Object

- Label: Consultation Note
- Plural: Consultation Notes
- Record Name: Auto Number or Text
- Enable Reports, Activities, Track History

The screenshot shows the 'Consultation Note' Custom Object Definition Edit page in Salesforce. The left sidebar contains a navigation menu with options like 'Details', 'Fields & Relationships', 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', 'Object Limits', 'Record Types', 'Related Lookup Filters', 'Restriction Rules', 'Scoping Rules', 'Object Access', and 'Triggers'. The main content area is titled 'Consultation Note' and includes a 'Custom Object Definition Edit' section with 'Save', 'Save & New', and 'Cancel' buttons. Below this is the 'Custom Object Information' section, which contains fields for 'Label' (Consultation Note), 'Plural Label' (Consultation Notes), 'Object Name' (Consultation_Note), and 'Description'. There are also checkboxes for 'Starts with vowel sound' and 'Context Sensitive Help Setting', and a dropdown for 'Content Name'.

Step 2: Create Fields

1. **Notes__c** – Long Text Area
2. **Diet_Plan__c** – Rich Text Area (Instead of Prescription)
3. **Consultation__c** – Master-Detail → Related To:
Consultation__c

Step 2: Relationships Summary

- **User__c** → **Consultation__c** → Master-Detail
- **Nutritionist__c** → **Consultation__c** → Lookup
- **Consultation__c** → **ConsultationNote__c** → Master-Detail

Step 3: Record Types

Consultation__c Record Types:

1. **User Submission**
 - Label: User Submission
 - Profile: User Profile

- Default Status: Pending

The screenshot shows the 'PERSONAL HEALTH TRACKER' setup interface. The top navigation bar includes 'Setup', 'Home', and 'Object Manager'. The main content area is titled 'Consultation' and shows the 'New Record Type' configuration for 'Consultation'. The interface is divided into a left sidebar with navigation options (Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, Object Access, Trippers) and a main configuration area.

Record Type Configuration:

- Existing Record Type:** --Master--
- Record Type Label:** User Submission
- Record Type Name:** User_Submission
- Description:** Generated when user submits the consultation request
- Active:** ☒

Make Available / Make Default:

Select Make Available to give users assigned to this profile the ability to create and clone records of this record type, or assign this record type to existing records. To make the new record type the default for a profile, select Make Default. Users assigned to this record type can still view and edit records associated with record types not available for their profiles.

Profile Name	Record Types Currently Available	Make Available	Make Default
Admin		☒	☐
Analytics Cloud Integration User		☒	☐

2. Attendant Review

- Label: Attendant Review
- Profile: Attendant Profile
- Status manually updated

3. Nutritionist Consultation

- Label: Nutritionist Consultation
- Profile: Nutritionist Profile
- Status depends on Attendant

Step 4: Page Layouts

1. Consultation__c Layouts

- **User Layout:** Basic request form (Status Read-only)
- **Attendant Layout:** Can update Status, assign Nutritionist
- **Nutritionist Layout:** Add Consultation Notes + Diet Plan

2. User__c Layout

- Basic Info (Name, Age, Mobile, Email)
- Health History
- Related List: Consultations

3. Nutritionist__c Layout

- Nutritionist Info (Name, Specialization, Contact)
- Related List: Consultations

4. ConsultationNote__c Layout

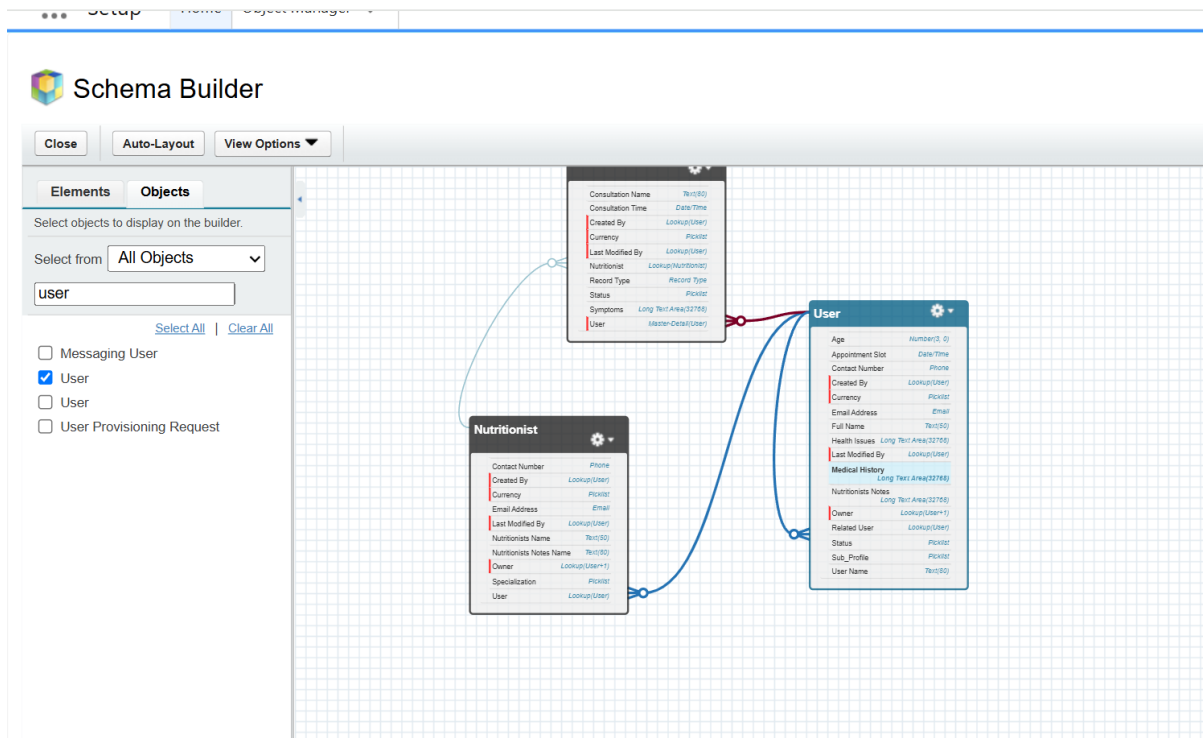
- Consultation Reference
- Notes + Diet Plan

Step 5: Compact Layouts

- **Consultation:** Status, Preferred Time, Nutritionist, User
- **User:** Name, Age, Mobile, Email
- **Nutritionist:** Name, Specialization, Phone
- **Consultation Note:** Note Number, Created Date, Notes

Schema Builder (Updated Relationships)

1. **User__c** → **Salesforce User** → Lookup
2. **User__c** → **Nutritionist__c** → Lookup (optional assignment)
3. **Consultation__c** → **User__c** → Master-Detail
4. **ConsultationNote__c** → **Consultation__c** → Master-Detail
5. **Optional Hierarchical** → **Salesforce User** → User
6. **Phase 4: Process Automation in Health Log Project**



Phase 4: Process Automation in Health Log Project

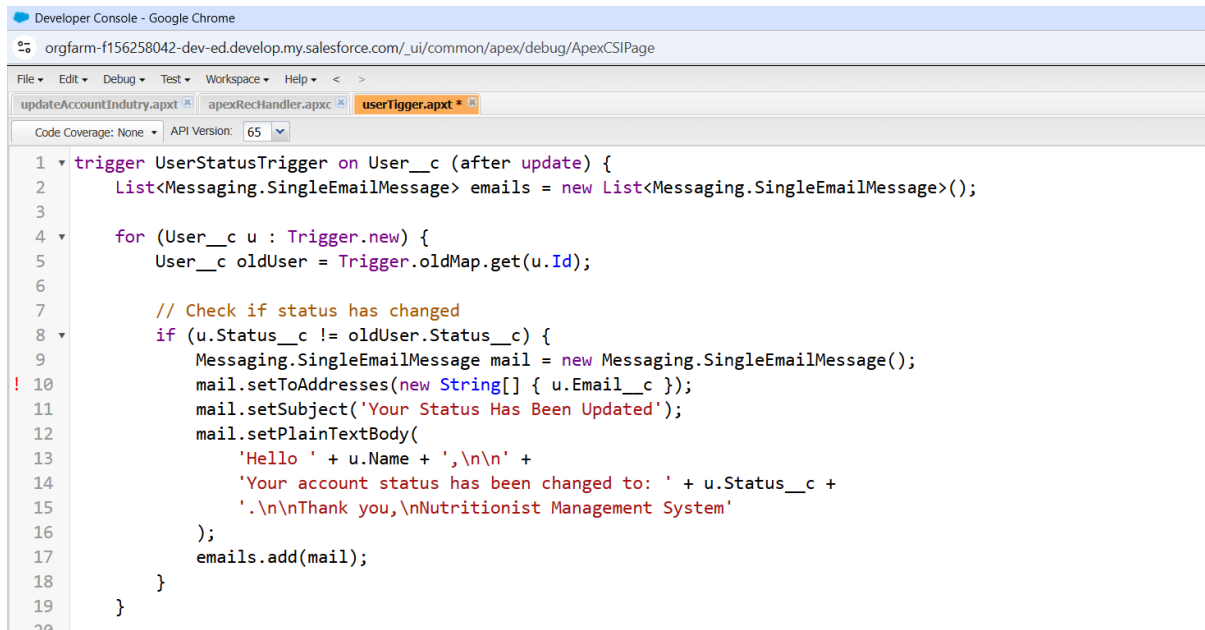
1.Tools & Features Used:

- **Validation Rules:**
Ensured correct input for email addresses and required fields in the **User** object.
- **Email Alerts:**
Implemented via **Apex triggers** when **User status** changes (Confirmed, Completed, Cancelled).
- **Field Updates / Workflow:**
Managed via user actions in the UI (**Users, Nutritionists, Attendants**) instead of using **Workflow Rules** or **Process Builder**.

Not Used / Optional:

- **Workflow Rules, Process Builder, Approval Process, Flow Builder, Tasks, Custom Notifications** –

Not required since all updates and notifications were handled via **Apex triggers** and **LWC actions**.



The screenshot shows the Salesforce Developer Console with the Apex trigger 'UserTrigger.apxt' open. The trigger is a 'trigger on User__c (after update)' and it sends an email notification when a user's status changes. The code is as follows:

```
1 trigger UserStatusTrigger on User__c (after update) {
2     List<Messaging.SingleEmailMessage> emails = new List<Messaging.SingleEmailMessage>();
3
4     for (User__c u : Trigger.new) {
5         User__c oldUser = Trigger.oldMap.get(u.Id);
6
7         // Check if status has changed
8         if (u.Status__c != oldUser.Status__c) {
9             Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
10            mail.setToAddresses(new String[] { u.Email__c });
11            mail.setSubject('Your Status Has Been Updated');
12            mail.setPlainTextBody(
13                'Hello ' + u.Name + ',\n\n' +
14                'Your account status has been changed to: ' + u.Status__c +
15                '.\n\nThank you,\nNutritionist Management System'
16            );
17            emails.add(mail);
18        }
19    }
20 }
```

Phase 5 : Apex Programming (Developer)

Step 1: Apex Trigger – Email Notification on User Status Change

Object: User__c

Trigger Events: after insert, after update

Purpose: To automatically send emails to users when their consultation status changes:

Status	Email Purpose
Confirmed	Notify the user that their nutrition consultation slot is confirmed.
Completed	Inform the user that the consultation has been completed successfully.
Cancelled	Notify the user that their selected slot is unavailable or cancelled.

```

1  trigger User_Triggers on User__c (before insert) {
2  if (Trigger.isAfter) {
3      if (Trigger.isInsert) {
4          UserTriggerHandler.handleAfterInsert(Trigger.new);
5      }
6      if (Trigger.isUpdate) {
7          UserTriggerHandler.handleAfterUpdate(Trigger.new, Trigger.oldMap);
8      }
9  }
10 }

```

Step 2: Trigger Design Pattern

- Separated trigger logic into a handler class (**UserTriggerHandler**) for better organization.
- Ensures **code reusability** and **easy maintenance** by keeping complex logic out of the trigger.
- Supports **bulk processing** using lists, avoiding SOQL/DML inside loops.
- Follows best practices to keep the trigger clean, scalable, and efficient.

```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt
Code Coverage: None API Version: 65

1 public class UserTriggerHandler {
2
3     // Handle after insert
4     public static void handleAfterInsert(List<User__c> newList) {
5         processEmails(newList, null);
6     }
7
8     // Handle after update
9     public static void handleAfterUpdate(List<User__c> newList, Map<Id, User__c> oldMap) {
10        processEmails(newList, oldMap);
11    }
12
13    // Common email logic
14    private static void processEmails(List<User__c> newList, Map<Id, User__c> oldMap) {
15        List<Messaging.SingleEmailMessage> emails = new List<Messaging.SingleEmailMessage>();
16
17        for (User__c userRecord : newList) {
18            String newStatus = userRecord.Status__c;
19            String oldStatus = (oldMap != null && oldMap.containsKey(userRecord.Id))
20                ? oldMap.get(userRecord.Id).Status__c
```

```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt
Code Coverage: None API Version: 65

17        for (User__c userRecord : newList) {
18            String newStatus = userRecord.Status__c;
19            String oldStatus = (oldMap != null && oldMap.containsKey(userRecord.Id))
20                ? oldMap.get(userRecord.Id).Status__c
21                : null;
22
23            // Send email only if status changed or newly inserted
24            if (newStatus != null && (oldStatus == null || newStatus != oldStatus)) {
25                Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
26                mail.setToAddresses(new String[] { userRecord.Email_Address__c });
27
28                if (newStatus == 'Confirmed') {
29                    mail.setSubject('Nutrition Consultation Confirmed');
30                    mail.setPlainTextBody(
31                        'Dear ' + userRecord.Name + ',\n\n' +
32                        'Your consultation with our nutritionist has been successfully confirmed.\n\n' +
33                        'Regards,\nNutrition Support Team'
34                    );
35
36                } else if (newStatus == 'Completed') {
```

```

File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt
Code Coverage: None API Version: 65
35
36 } else if (newStatus == 'Completed') {
37     mail.setSubject('Consultation Completed Successfully');
38     mail.setPlainTextBody(
39         'Dear ' + userRecord.Name + ',\n\n' +
40         'Your consultation session has been successfully completed.\n\n' +
41         'Regards,\nNutrition Support Team'
42     );
43
44 } else if (newStatus == 'Cancelled') {
45     mail.setSubject('Consultation Slot Unavailable');
46     mail.setPlainTextBody(
47         'Dear ' + userRecord.Name + ',\n\n' +
48         'Unfortunately, your selected consultation slot is no longer available. ' +
49         'Please rebook at your convenience.\n\n' +
50         'Regards,\nNutrition Support Team'
51     );
52 }
53
54 emails.add(mail);

```

```

File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt
Code Coverage: None API Version: 65
44 } else if (newStatus == 'Cancelled') {
45     mail.setSubject('Consultation Slot Unavailable');
46     mail.setPlainTextBody(
47         'Dear ' + userRecord.Name + ',\n\n' +
48         'Unfortunately, your selected consultation slot is no longer available. ' +
49         'Please rebook at your convenience.\n\n' +
50         'Regards,\nNutrition Support Team'
51     );
52 }
53
54 emails.add(mail);
55 }
56 }
57
58 if (!emails.isEmpty()) {
59     Messaging.sendEmail(emails);
60 }
61 }
62 }
63
Logs Tests Checkpoints Query Editor View State Progress Problems
Name Line Problem

```

Step 3: SOQL & SOSL

- **SOQL:** Used to fetch User__c records from the database for sending emails or updating related information (e.g., retrieving email addresses or linked nutritionist details).

Example:

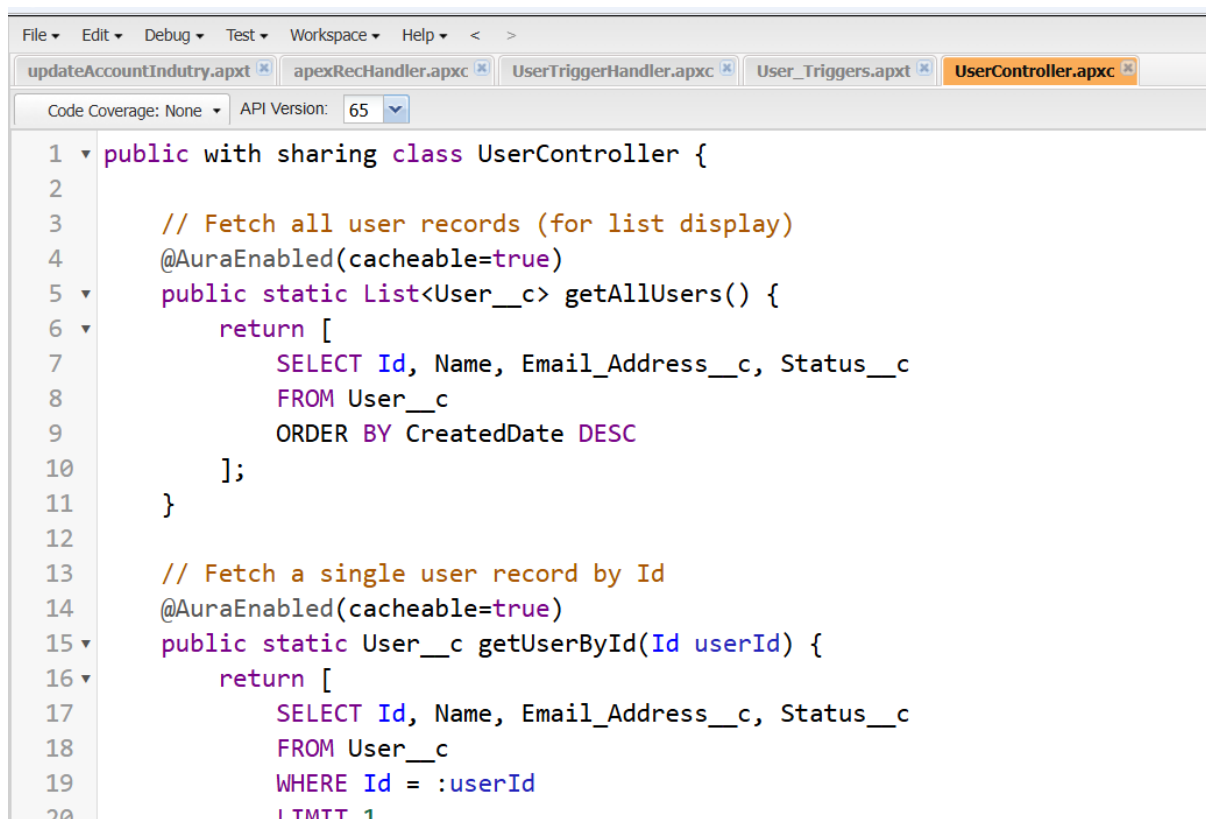
- List<User__c> userList = [SELECT Id, Name, Email_Address__c, Status__c FROM User__c];

Step 4: Collections (List, Set, Map)

- **List:** Used to store all outgoing emails and send them in one bulk call using Messaging.sendEmail().
 - **Set / Map:** Can be used to avoid duplicate email addresses or to map record Ids to old values for quick lookup.
 - This ensures **efficient bulk processing** and prevents **governor limit** errors.
-

Step 5: Control Statements

- **If / Else:** Used to check each user's consultation status (Confirmed, Completed, or Cancelled) and decide which email to send.
- **Loops:** Used to iterate through multiple User__c records in a single trigger execution.



```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt UserController.apxc
Code Coverage: None API Version: 65
1 public with sharing class UserController {
2
3     // Fetch all user records (for list display)
4     @AuraEnabled(cacheable=true)
5     public static List<User__c> getAllUsers() {
6         return [
7             SELECT Id, Name, Email_Address__c, Status__c
8             FROM User__c
9             ORDER BY CreatedDate DESC
10        ];
11    }
12
13    // Fetch a single user record by Id
14    @AuraEnabled(cacheable=true)
15    public static User__c getUserById(Id userId) {
16        return [
17            SELECT Id, Name, Email_Address__c, Status__c
18            FROM User__c
19            WHERE Id = :userId
20        ];
21    }
22 }
```

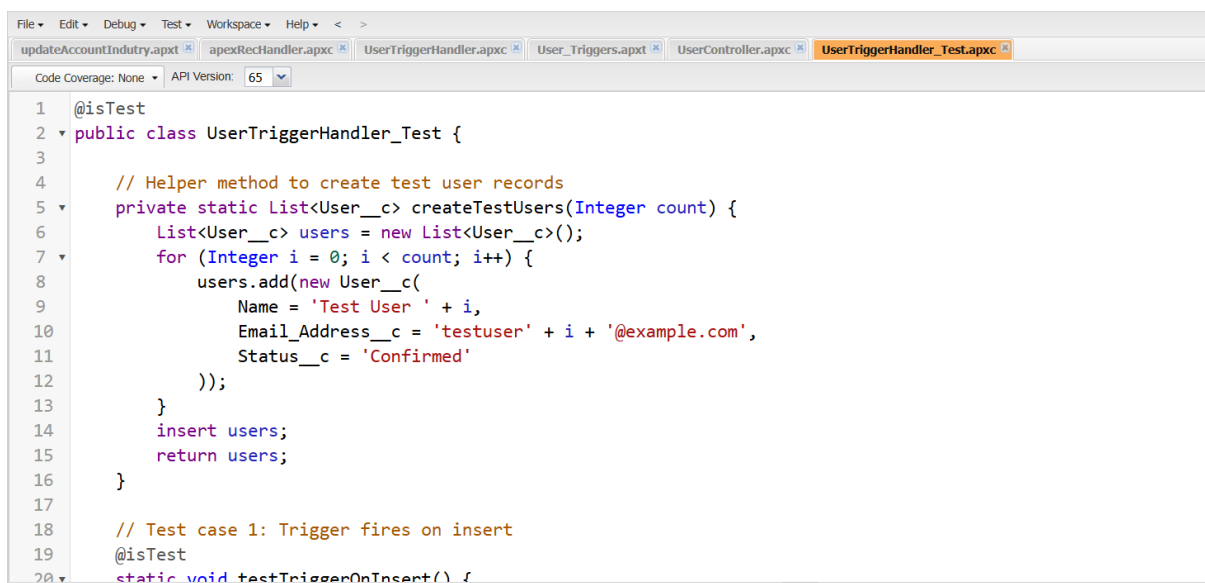
```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt UserController.apxc
Code Coverage: None API Version: 65
21     };
22     }
23
24     // Create or update a user record
25     @AuraEnabled
26     public static void saveUser(User__c userRecord) {
27         if (userRecord.Id == null) {
28             insert userRecord; // new record
29         } else {
30             update userRecord; // existing record
31         }
32     }
33
34     // Delete a user record
35     @AuraEnabled
36     public static void deleteUser(Id userId) {
37         delete [SELECT Id FROM User__c WHERE Id = :userId];
38     }
39
40     // Filter users by status (Confirmed, Completed, Cancelled)
```

```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRecHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt UserController.apxc
Code Coverage: None API Version: 65
32     }
33
34     // Delete a user record
35     @AuraEnabled
36     public static void deleteUser(Id userId) {
37         delete [SELECT Id FROM User__c WHERE Id = :userId];
38     }
39
40     // Filter users by status (Confirmed, Completed, Cancelled)
41     @AuraEnabled(cacheable=true)
42     public static List<User__c> getUsersByStatus(String statusValue) {
43         return [
44             SELECT Id, Name, Email_Address__c, Status__c
45             FROM User__c
46             WHERE Status__c = :statusValue
47             ORDER BY Name
48         ];
49     }
50 }
51
Logs Tests Checkpoints Query Editor View State Progress Problems
Name Line Problem
```

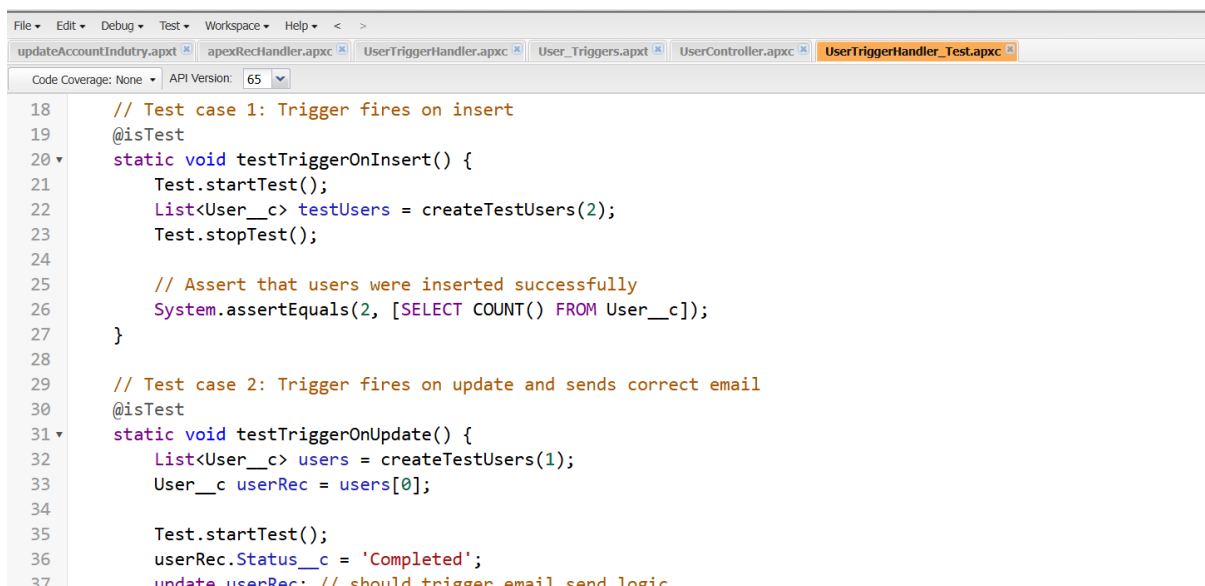

Step 6: Test Classes

Purpose

- Validate that the **User Trigger** and **Handler** work correctly.
- Ensure that **emails are sent** when the Status__c field changes.
- Verify **bulk record processing** to comply with Salesforce governor limits.
- Achieve **100% code coverage** for deployment to production.



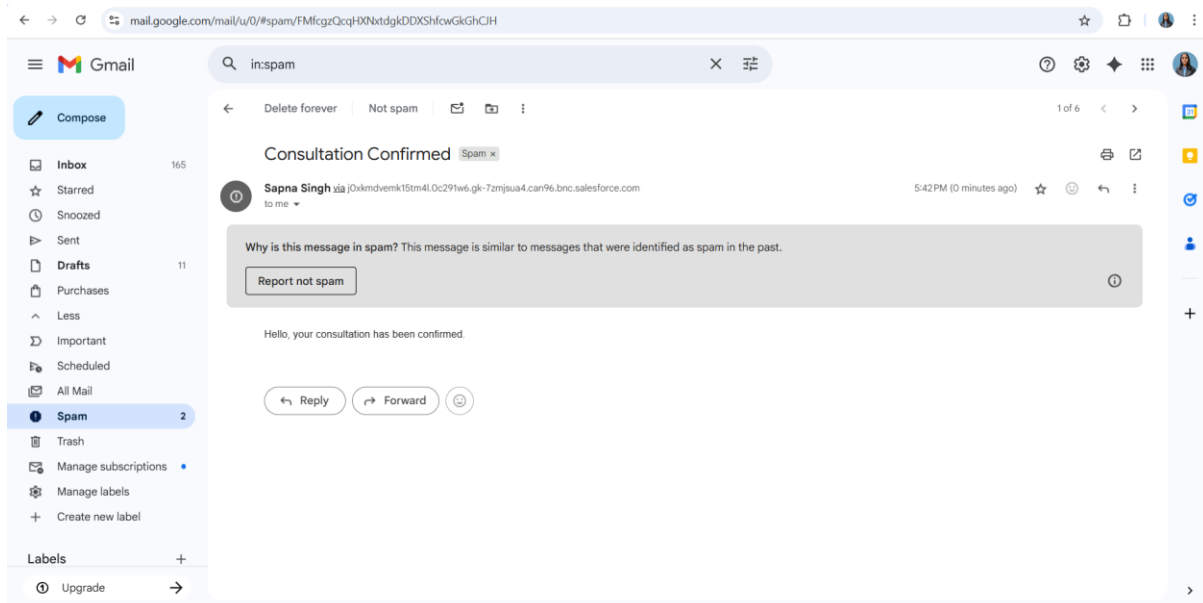
```
1 @isTest
2 public class UserTriggerHandler_Test {
3
4     // Helper method to create test user records
5     private static List<User__c> createTestUsers(Integer count) {
6         List<User__c> users = new List<User__c>();
7         for (Integer i = 0; i < count; i++) {
8             users.add(new User__c(
9                 Name = 'Test User ' + i,
10                Email_Address__c = 'testuser' + i + '@example.com',
11                Status__c = 'Confirmed'
12            ));
13        }
14        insert users;
15        return users;
16    }
17
18    // Test case 1: Trigger fires on insert
19    @isTest
20    static void testTriggerOnInsert() {
```



```
18    // Test case 1: Trigger fires on insert
19    @isTest
20    static void testTriggerOnInsert() {
21        Test.startTest();
22        List<User__c> testUsers = createTestUsers(2);
23        Test.stopTest();
24
25        // Assert that users were inserted successfully
26        System.assertEquals(2, [SELECT COUNT() FROM User__c]);
27    }
28
29    // Test case 2: Trigger fires on update and sends correct email
30    @isTest
31    static void testTriggerOnUpdate() {
32        List<User__c> users = createTestUsers(1);
33        User__c userRec = users[0];
34
35        Test.startTest();
36        userRec.Status__c = 'Completed';
37        update userRec; // should trigger email send logic
```

```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRechHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt UserController.apxc UserTriggerHandler_Test.apxc
Code Coverage: None API Version: 65
34
35     Test.startTest();
36     userRec.Status__c = 'Completed';
37     update userRec; // should trigger email send logic
38     Test.stopTest();
39
40     // Validate that status updated successfully
41     User__c updated = [SELECT Status__c FROM User__c WHERE Id = :userRec.Id];
42     System.assertEquals('Completed', updated.Status__c);
43 }
44
45 // Test case 3: Bulk processing (multiple updates)
46 @isTest
47 static void testBulkProcessing() {
48     List<User__c> bulkUsers = createTestUsers(10);
49
50     // Change statuses in bulk
51     for (User__c u : bulkUsers) {
52         u.Status__c = 'Cancelled';
53     }
```

```
File Edit Debug Test Workspace Help < >
updateAccountIndustry.apxt apexRechHandler.apxc UserTriggerHandler.apxc User_Triggers.apxt UserController.apxc UserTriggerHandler_Test.apxc
Code Coverage: None API Version: 65
45 // Test case 3: Bulk processing (multiple updates)
46 @isTest
47 static void testBulkProcessing() {
48     List<User__c> bulkUsers = createTestUsers(10);
49
50     // Change statuses in bulk
51     for (User__c u : bulkUsers) {
52         u.Status__c = 'Cancelled';
53     }
54
55     Test.startTest();
56     update bulkUsers; // Trigger should handle all in bulk
57     Test.stopTest();
58
59     // Verify that all statuses updated
60     Integer cancelledCount = [SELECT COUNT() FROM User__c WHERE Status__c = 'Cancelled'];
61     System.assertEquals(10, cancelledCount);
62 }
63 }
64
Logs Tests Checkpoints Query Editor View State Progress Problems
Name Line Problem
```



7. Exception Handling

- Implemented **try-catch blocks** to handle potential exceptions during **DML operations** or **email sending**.
- Ensures that if **one record fails** (e.g., email fails), the trigger **continues processing other records** without stopping.
- Helps maintain **data integrity** and **smooth execution** of bulk operations.

Phase 6: User Interface Development

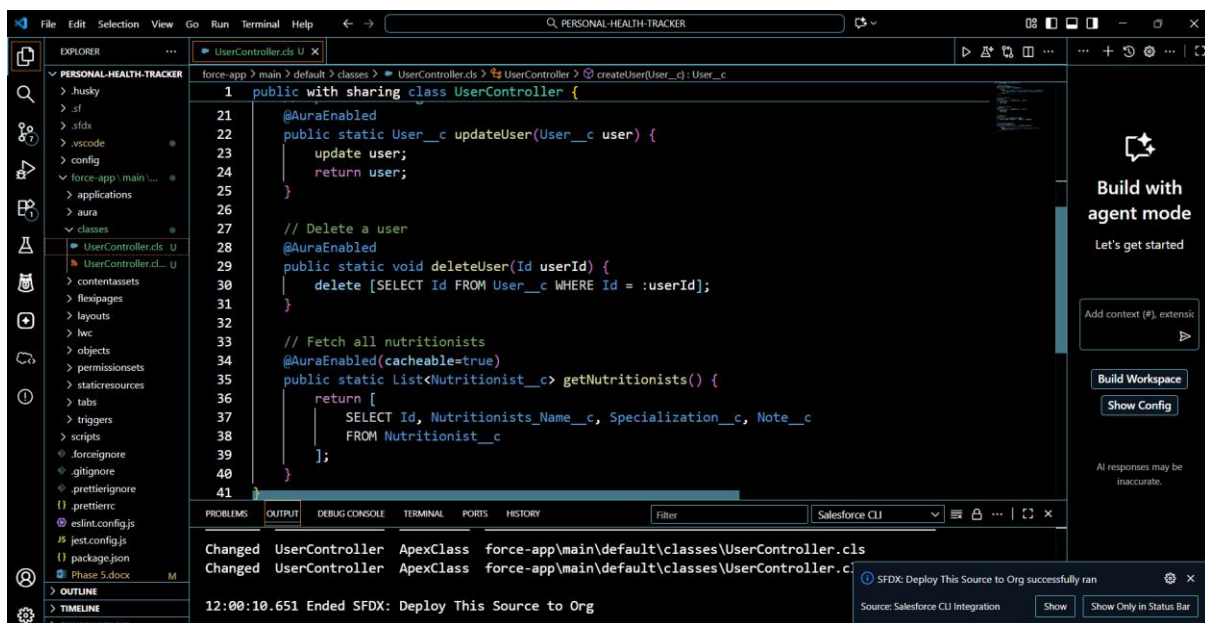
1. Tools & Features Used

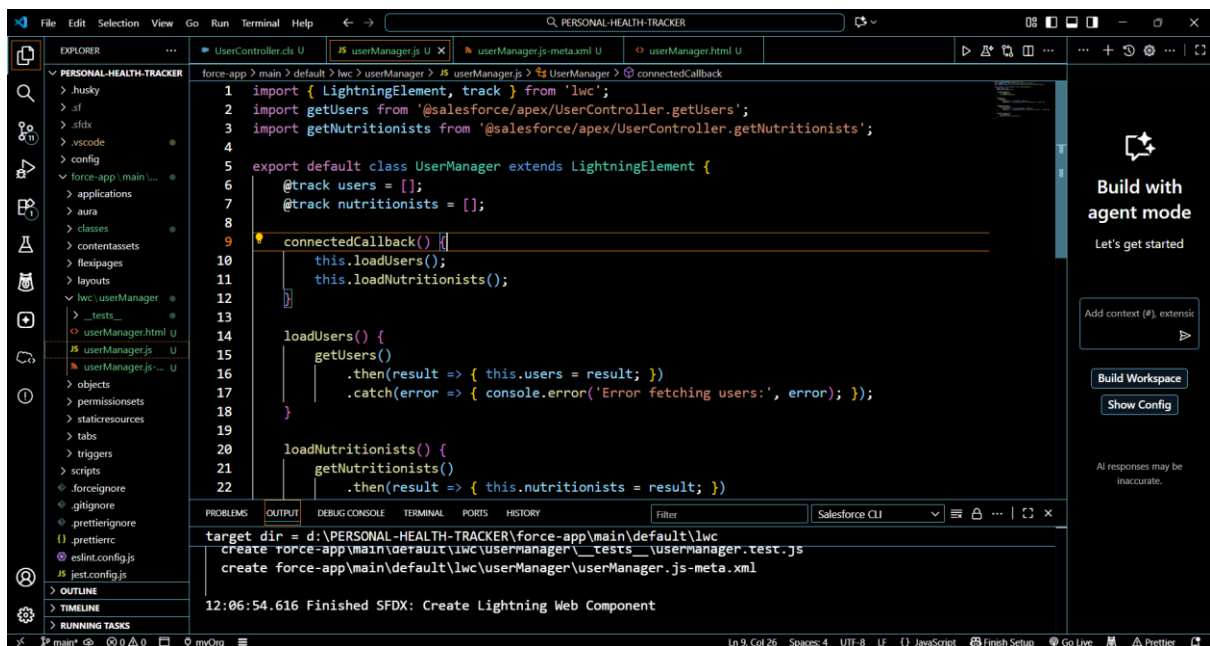
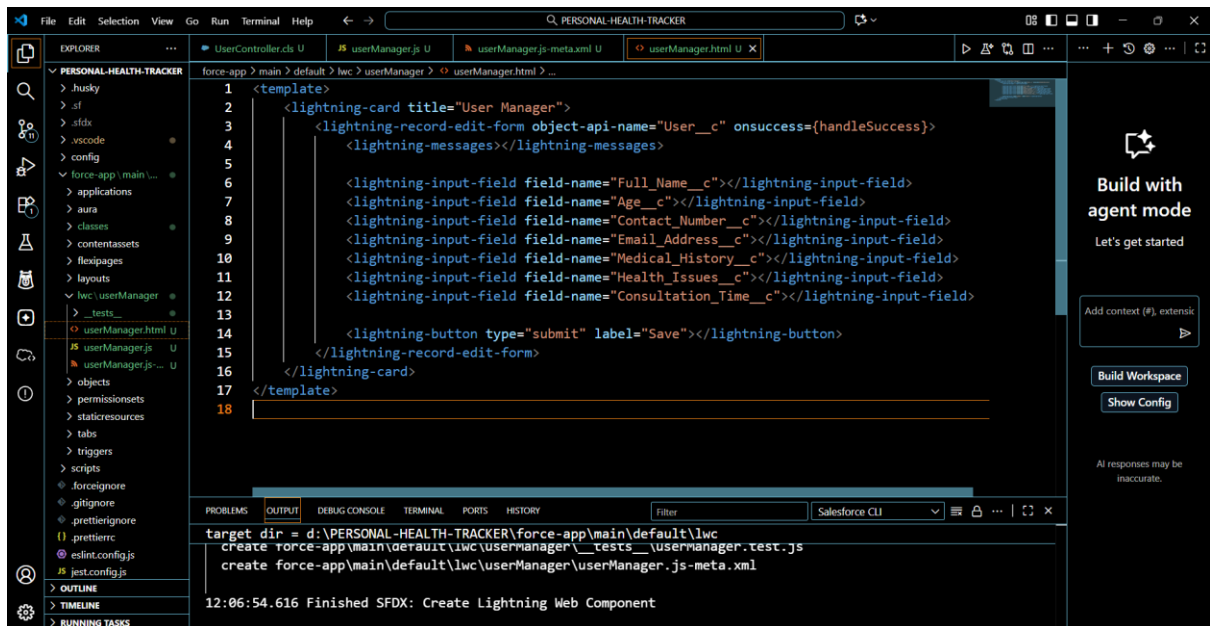
- **Lightning App Builder:** Customized record pages, tabs, and home pages for Users, Nutritionists, and Attendants.
- **Lightning Web Components (LWC):** Built User Manager component to create, view, and update user records.
- **Apex with LWC:** Connected LWCs to **UserController** for fetching and updating data.
- **Custom Events in LWC:** Enabled communication between child and parent components.

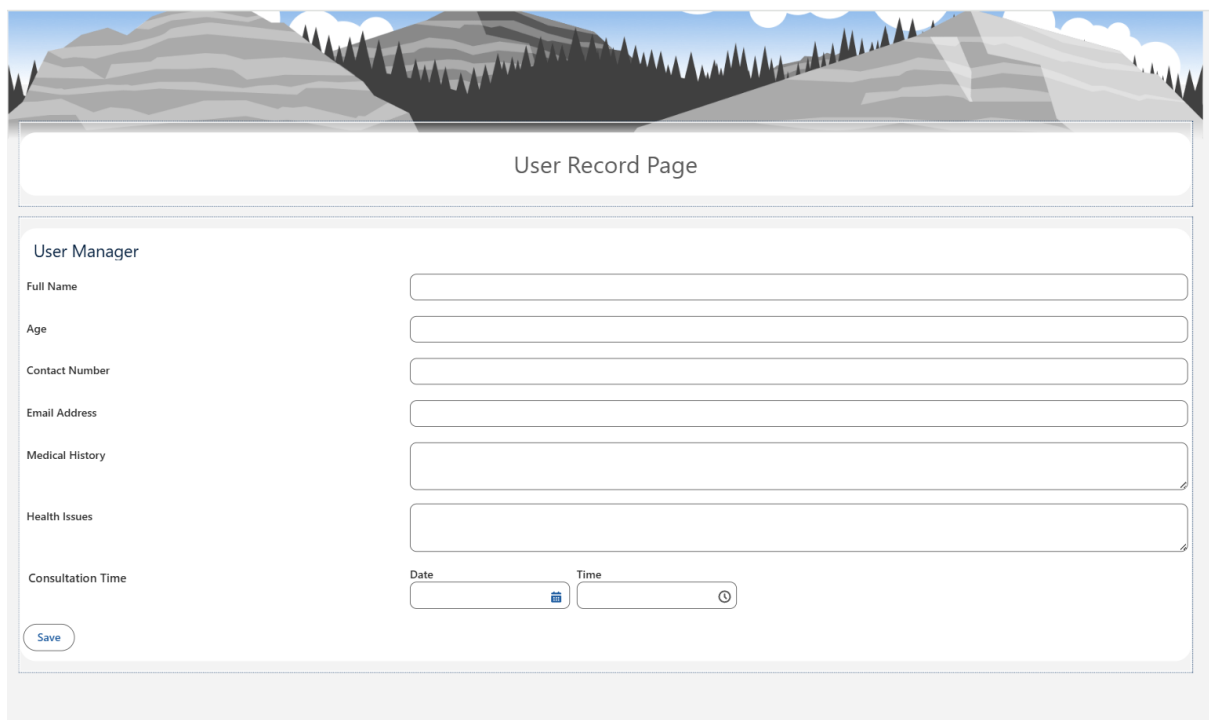
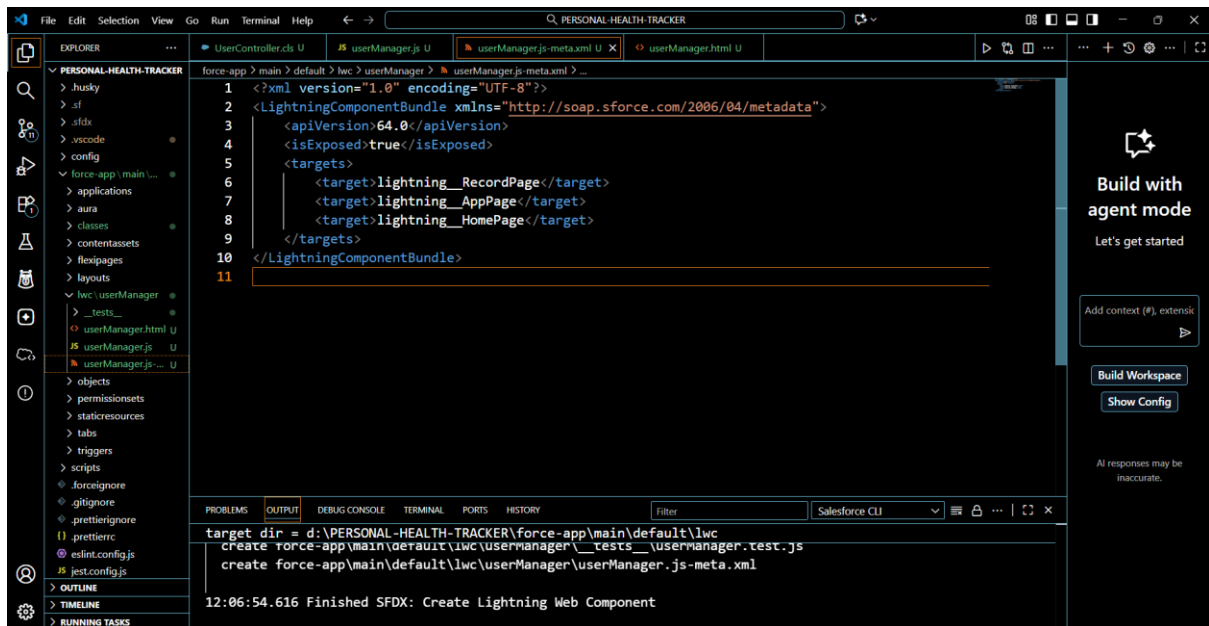
- **Wire Adapters:** Reactively fetched records and user profiles.
- **Imperative Apex Calls:** Handled actions like creating users, updating status, deleting records, and saving nutritionist notes.

2. Project Relevance / Implementation

- **User UI:** Allows users to submit:
 - Personal details (Name, Age, Mobile, Email)
 - Medical history
 - Symptoms
 - Preferred consultation time







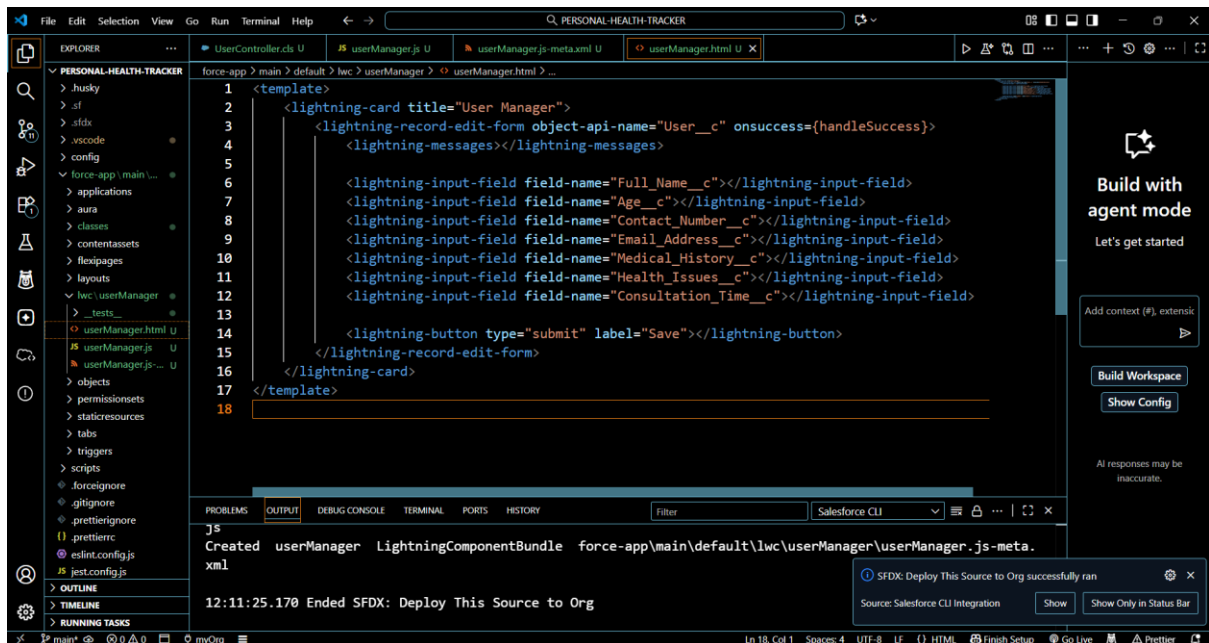
Phase 8: Data Management & Deployment

I managed data and deployment as follows:

- **Change Sets / VS Code / SFDX:** Development was done directly in a free **Developer Edition org**. LWCs and Apex

components were deployed using **Salesforce Extensions for VS Code**. No sandbox-to-production deployment was required.

- **Unmanaged Packages:** All objects, LWCs, and triggers were custom **unmanaged components** created directly in the org.



Phase 9: Reporting, Dashboards & Security Review

Reporting and security were managed to ensure proper access control:

- **Dynamic Dashboards:** Not implemented; access is controlled via user sub-profiles.
- **Sharing Settings & Field-Level Security:** Managed using sub-profile logic. Users see only their own records, while nutritionists and attendants can access all relevant user records.
- **Session Settings & Login IP Ranges:** Default org security settings were sufficient.